

0720 update — gate_v2 与 h_j probe 的组合探索

承接 0713update.md：那份文档第八节留了一个"下一步"——adaptive_lambda_rerank（学一个逐 hop 自适应权重，替代固定 λ ）当时还没有真实结果。这次把这条线走完了，中途踩了三次坑、最终这个"自适应权重"方向本身被证伪；但过程中顺手把 gate_v2 用 lr_rerank 模式重新跑了一遍，拿到了这个 session 里第一次四项指标同时全面超过 baseline 的结果，并在此基础上把 gate_v2 和 h_j probe 这两个已经分别验证过的信号真正组合了起来，拿到了目前为止最好的端到端结果。

一、K=3/4 数据增强：真实用上了 train_nl_enhance.jsonl

adaptive_lambda_rerank 训练 λ 模型需要真实候选池数据，复用 rerank_listwise_contrastive/build_topk_pools.py 产出的池子时，发现跟 error_propagation_probe 那次一样的老问题——MuSiQue 本身 K 分布不均衡，train 里 K=2 占大头（61.7%）。用 data/decompose/musique/gt/train_nl_enhance.jsonl（当初为了这个目的准备好、但一直没人用过的 LLM 改写子问题，专门覆盖 K=3/4，target_k: [3, 4]）跑了一遍真实 BGE 检索，造出额外的真实候选池（不是复制数据，是换一个措辞重新检索一遍，天然不同的候选池），补进训练数据：

```
补齐前（原始 pool）：K=2 61.7% / K=3 28.2% / K=4 10.1%
补齐后（59857 个 pool）：K=2 48.0% (28752) / K=3 38.5% (23013) / K=4
13.5% (8092)
```

K=3/4 占比明显上来了。这批数据后续用于 adaptive_lambda_rerank 的训练。

二、adaptive_lambda_rerank：自适应权重三次失败，方向本身被证伪

设计思路：不再用固定 λ ，训一个很小的模型，输入这一跳的上下文（q_main_emb/h_prev/expanded_q_emb），输出这一跳专属的 λ ，同一个 λ 用在这一跳全部候选上（只决定"这一跳该多信 cosine 还是多信 probe"，不针对具体候选）。loss 是 listwise softmax（gold 排第一），dev 上跟两个固定基线对比 gold 排名。

第一次：无界输出， λ 直接跑飞

softplus 激活没有上界，训练发现只要一直调大 λ 就能持续压低 loss，没有任何机制拦着：

```
Dev learned lambda: mean=9.9932  std=2.1244  min=0.0347  max=17.5034
```

	recall@1	recall@3	mrr
纯 emb_score ($\lambda=0$)	0.8067	0.9396	0.8778
固定 $\lambda=0.25$ （生产默认）	0.8601	0.9635	0.9145
学出来的 λ	0.701	0.877	0.8015

学出来的还不如纯 cosine，全面倒退。

第二次：加上界 ($\lambda_{\max} \cdot \text{sigmoid}$)，结果卡死不动

给 λ 设了 $[0, 2]$ 的硬边界。但训练完全学不动，loss 卡在 ~ 1.86 反复横跳好几个 epoch 不降。查了真实数据才发现根因： h_{prev} (Llama 原始隐藏状态) 每行 L2 范数均值 ≈ 149 ，而 $q_{\text{main_emb}} / \text{expanded_q_emb}$ (BGE 归一化过) 范数是 1.0——尺度差 150 倍，拼进同一个特征向量喂给 MLP，不管最后用什么激活函数，都会把预激活值推到极端：用无界 softplus 会跑飞（第一次的坑），用有界 sigmoid 会直接饱和、梯度趋近于零（这一次的坑）——两次失败是同一个根因，只是换了个症状。

把 h_{prev} 在过投影层之前做 L2 归一化后，loss 恢复正常下降（8 步内 $0.39 \rightarrow 0.03$ ，真实数据验证过 梯度健康）。重新跑完整训练，loss 这次能稳步下降到 epoch 23 左右的 0.78 (vs 之前卡死的 1.86)，但最终结果是塌缩成常数：

```
Dev learned lambda: mean=2.0000  std=0.0000  min=2.0000  max=2.0000
```

模型对所有跳都固定输出上界 2.0，等于变相退化成“固定 $\lambda=2.0$ ”，而且这个值本身就不合理 (recall@1=0.7634，仍然不如固定 0.25 的 0.8601) ——listwise loss 的梯度在整个范围内几乎都指向“ λ 越大越好”（因为多扣一点 abnormal_score，大部分情况下确实能把 gold 和非 gold 分得更开），没有任何“够了别再推”的机制，只要给它空间就会一路顶到上限。

第三次：换成有界凸组合 + z-score 校准，训练终于健康，但效果依然不够

把“学一个开放系数”换成“学一个 $[0,1]$ 的混合比例 (gate) ”： emb_score 和 $-\text{abnormal_score}$ 各自在池内做 z-score 归一化（消除尺度不一致），再用 $\text{gate} \cdot \text{emb_z} + (1 - \text{gate}) \cdot \text{abn_z}$ 做凸组合——凸组合的权重加起来必须是 1，结构上就杜绝了“某一方无限压过另一方”。

这次训练终于健康了：

```
Dev learned gate: mean=0.4817  std=0.3788  min=0.0000  max=1.0000
```

gate 有真实方差、覆盖 0~1 整个范围，说明模型真的在按 hop 区分，不再是常数。但最终结果：

	recall@1	recall@3	mrr
纯 emb_score	0.8067	0.9396	0.8778
固定 $\lambda=0.25$	0.8601	0.9635	0.9145
学出来的 gate	0.8042	0.9318	0.8736

依然没有超过固定 0.25，甚至比纯 emb_score 还略差。训练过程的三个 bug 全部修完之后，这条路线本身的天花板还是没打破——根本原因：这个模型的输入全部是检索发生之前就定好的文本上下文（主问题、走到这里的推理状态、当前子问题），它从头到尾没看过这一跳实际召回出来的候选长什么样，单靠预判“这是个什么样的问题”去猜权重，信息量不够。结论：context-only 的自适应加权这条路线本身有问题，不是训练没调好——需要能看到实际候选打分的信号，而不是检索前的盲猜。

一个中间验证：RRF（不需要训练）

在放弃"学权重"之前，用不需要训练的 Reciprocal Rank Fusion（倒数排名融合）试了一版，只融合 emb_score 和 h_j-probe 两路（当时还没有 gate_v2 的候选级数据）：

	recall@1	recall@3	mrr
纯 emb_score	0.8067	0.9396	0.8778
学出来的 gate	0.8042	0.9318	0.8736
RRF (k=10)	0.8304	0.9539	0.8959
固定 $\lambda=0.25$	0.8601	0.9635	0.9145

RRF 比纯 emb_score 和学出来的 gate 都好，但还是没打过固定 $\lambda=0.25$ ——这一版只融合了两路，没有 gate_v2，后面第五节会看到加入 gate_v2 之后 RRF 的表现（依然不如直接加权）。

三、gate_v2 换成 lr_rerank：第一次四项指标同时全面超过 baseline

0713update.md 第一节验证 gate_v2 时用的是 gated_rule_a（二元触发，gate 分数超阈值才扩池，gate 本身不参与排序）。这次直接用生产脚本 run_retrieval_exp_wavefront.py --methods lr_rerank（连续混合公式 emb_score - $\lambda \cdot$ gate_score，跟 h_j-probe 那边用的是同一种公式结构）重新跑了一遍，其它参数保持跟之前 gated_rule_a 那次一致（lambda_lr=0.25、decompose_mode=bart_decompose、artifacts_dir=gate/artifacts_pooled_v2）：

	recall@1	chain_recall@1	EM	F1
baseline gated_rule_a（31层）	0.6513	0.4233	0.4079	0.5012
gate_v2 gated_rule_a（混层，二元触发）	0.6565	0.4290	持平	持平
gate_v2 lr_rerank（连续混合）	0.6627	0.4357	0.4100	0.5045

这是这个 session 里第一次四项指标同时全面超过 baseline——之前 gate_v2 的二元触发版本只是 检索指标涨、答案指标持平；这次换成连续混合公式后，四项全涨，虽然每项涨幅不算夸张，但方向一致，没有此消彼长。说明"连续混合"这个公式结构本身，比 gate_v2 原来的二元触发用法更好，跟 h_j-probe 那边"混合比单独用/单独触发更稳"的经验一致。

四、gate_v2 和 h_j-probe 的重叠分析：证明两者互补，值得组合

在决定要不要花力气把两个信号真正组合起来之前，先直接比了一下两边的真实结果，看"谁做对的样例" 重不重合——如果完全重合就没必要合并。用上一节的 gate_v2 lr_rerank 结果和之前的 h_j-probe weighted（ $\lambda=0.25$ ）结果，按 id 对齐比较（2417 个共同样例）：

chain_recall1（检索链路）：

两个都对： 911 (37.7%)

两个都错： 1186 (49.1%)

只 gate_v2 对： 142 (5.9%)
只 h_j-probe 对： 178 (7.4%)

答案 EM：

两个都对： 797 (33.0%)
两个都错： 1273 (52.7%)
只 gate_v2 对： 194 (8.0%)
只 h_j-probe 对： 153 (6.3%)

两项指标上都有 **13~14%** 的样例是"只有一个方法做对"——不是谁包含谁，是两边各自独立在不同样例 上有效。理想情况下（完美挑对）EM 上限能到 47.3%，比单用任何一个（约 39~41%）都高出不少。**结论：两个信号不是同一件事的两种说法，值得合并。**

五、combined_gate_rerank：三种投票方式对比，简单加权求和胜出

新建了 combined_gate_rerank/wavefront_combined_gate.py，两个信号在同一次批量前向传播里一起打分（gate_v2 按自己的 j 用 15/23 层，probe 固定 31 层，一次 batch_last_hidden 把这几层 都问出来，不需要跑两次 Llama），支持三种投票方式：

投票方式	公式	recall@1/oracle	chain 相关	EM	F1
baseline	—	0.6513	0.4233	0.4079	0.5012
gate_v2 lr_rerank (单独)	—	0.6627	0.4357	0.4100	0.5045
h_j-probe weighted (单独)	—	0.6735	0.4659	0.393	0.4816
联合 sum	emb - $\lambda_1 \cdot \text{gate_v2}$ - $\lambda_2 \cdot \text{probe}$, λ 各 0.25 (复用各自已验证的值)	0.6869	0.4894	0.4141	0.5089
联合 veto	任一模型判定异常 ($p>0.5$) 就否决，剩下按 cosine 排	没有效果			
联合 rrf	两边各自完整重排后，融合名次 (不是分数)	0.6705	0.4564	0.3922	0.4822

sum（最简单的固定权重求和）是三种里唯一全面超过之前所有单独方法的，**veto** 和更"精巧"的 **rrf**（不需要做尺度校准，只融合排名先后）反而都不如它——**rrf** 检索侧还行但答案侧甚至不如 baseline，大概率是因为它只看名次、丢掉了"有多确定"这个置信度信息，而 **sum** 保留了原始分数的 幅度差异。这也再次印证了第二节的经验：**越简单直接的固定权重组合，比越"聪明"的方案更稳。**

协同作用验证：不是"选更强的那个"，是真的互补

用第四节同样的方法，检查 `sum` 模式相对两个单独方法的"重叠恢复率"：

只有 `gate_v2` 单独对的 142 个样例：联合模型保住了 47.2%
只有 `h_j-probe` 单独对的 178 个样例：联合模型保住了 78.1%
两个单独都错的 1186 个样例：联合模型新修好了 4.9% (约 58 个)

最后一行最关键——这 58 个样例，`gate_v2` 和 `h_j-probe` 单独看都是错的，只有两个信号组合起来才做对，不是"联合模型学会了偏向更准的那个"，是真的从组合信息里挖出了单独任何一个都拿不到的正确答案。前两行也说明协同是双向的——没有偏废任何一方。**结论：确认存在真正的协同效应，不只是均值意义上的提升。**

六、下一步（进行中）：置信度加权

`sum` 模式虽然是目前最好的，但离"理想情况下完美挑对"的 47.3% EM 上限还有明显差距——用固定权重到处都一样地混，没有针对"这一跳该多信哪个"做任何区分。下一步想法：不训练模型，用两个信号各自在这一跳候选池内部的打分标准差（越分散说明这次判断越果断、越有信息量）算一个动态权重：

```
w_gate = gate_v2置信度 / (gate_v2置信度 + probe置信度)
w_probe = 1 - w_gate
最终分数 = emb_score - (2×0.25×w_gate)·gate_v2_score -
(2×0.25×w_probe)·probe_score
```

两边置信度相近时自动退化成 `sum` 模式（各 0.25），只有某一跳里两边"果断程度"差很多时才真的倾向更自信的那个——不用训练，不会重演第二节那三次"学出来的权重不可控"的坑。

验证这个不需要重跑端到端：`adaptive_lambda_rerank` 已经有 `gate_v2` 和 `h_j-probe` 各自完整候选池的打分基础设施，只差给现有的 `musique_{train,dev}_lambda_data.npz` 补一列 `cand_gate_v2_score`（`add_gate_v2_scores.py` 已写好，`pool_id` 对齐检查在真实 train/dev 数据上验证过，逻辑用真实 `gate_v2` 模型 + 正确维度的测试数据跑通过），补完之后可以在 CPU 上直接用 `report_gold_rank` 那套现成方法离线对比，不需要每次改个想法就重跑一次完整的 GPU 端到端流程。

这一步（真正过 `Llama` 给候选池补 `gate_v2` 分数）还没有实际跑，是下一次更新要补的结果。